

TEKNOLOGI IOT II
Modul 4 (LATIHAN 4.6)



DISUSUN OLEH:

NAMA : DEA ANGGRAENI
NPM : 25782036
KELAS :3B
JURUSAN :TEKNOLOGI INFORMASI
PRODI :TEKNOLOGI REKAYASA
INTERNET

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, SPd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026/2027

4.6 Latihan

Latihan 1 (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawaban :

WebSockets jauh lebih hemat data dan bebas kedip (*flickering*) karena menggunakan satu koneksi pipa terbuka (*full-duplex*) yang hanya mengirim paket data kecil secara instan saat ada pembaruan. Sebaliknya, *Meta Refresh HTTP* harus mengunduh ulang seluruh halaman web dari awal secara terus-menerus setiap beberapa detik, sehingga memboroskan *bandwidth* dan membuat layar berkedip

Latihan 2 (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan *NodeMCU*) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Jawaban :

`delay(3000)` di dalam `void loop()` meliputi:

- **Pemblokiran Total CPU (*Blocking*):** Fungsi `delay(3000)` membuat mikrokontroler (*NodeMCU*) berhenti total dan "tidur" selama 3 detik. Selama waktu tersebut, CPU tidak bisa mengeksekusi proses jaringan sama sekali.
- **Koneksi WebSocket Terputus (*Timeout/Disconnect*):** Protokol *WebSocket* membutuhkan pemeliharaan koneksi berkala (seperti *ping/pong* atau *heartbeat*). Karena CPU tertidur selama 3 detik, *NodeMCU* gagal merespons sinyal jaringan tepat waktu sehingga server/browser mengira perangkat terputus.
- **Sistem Sangat Lambat / *Lag*:** Permintaan tombol dari browser (misalnya menyalakan LED) yang diklik pengguna tidak akan direspons instan, melainkan harus menunggu jeda `delay` selesai.
- **Risiko *Watchdog Timer (WDT) Reset*:** Pada *ESP8266/NodeMCU*, jika proses utama tertahan terlalu lama tanpa mengeksekusi fungsi sistem latar belakang (seperti *Wi-Fi stack*), sistem keamanan internal akan menganggap program mengalami *hang* (macet) dan secara paksa melakukan *reboot* (restart) otomatis pada modul

Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawaban:

tempValue — Target ID untuk menyuntikkan dan memperbarui data angka suhu secara *real-time*.

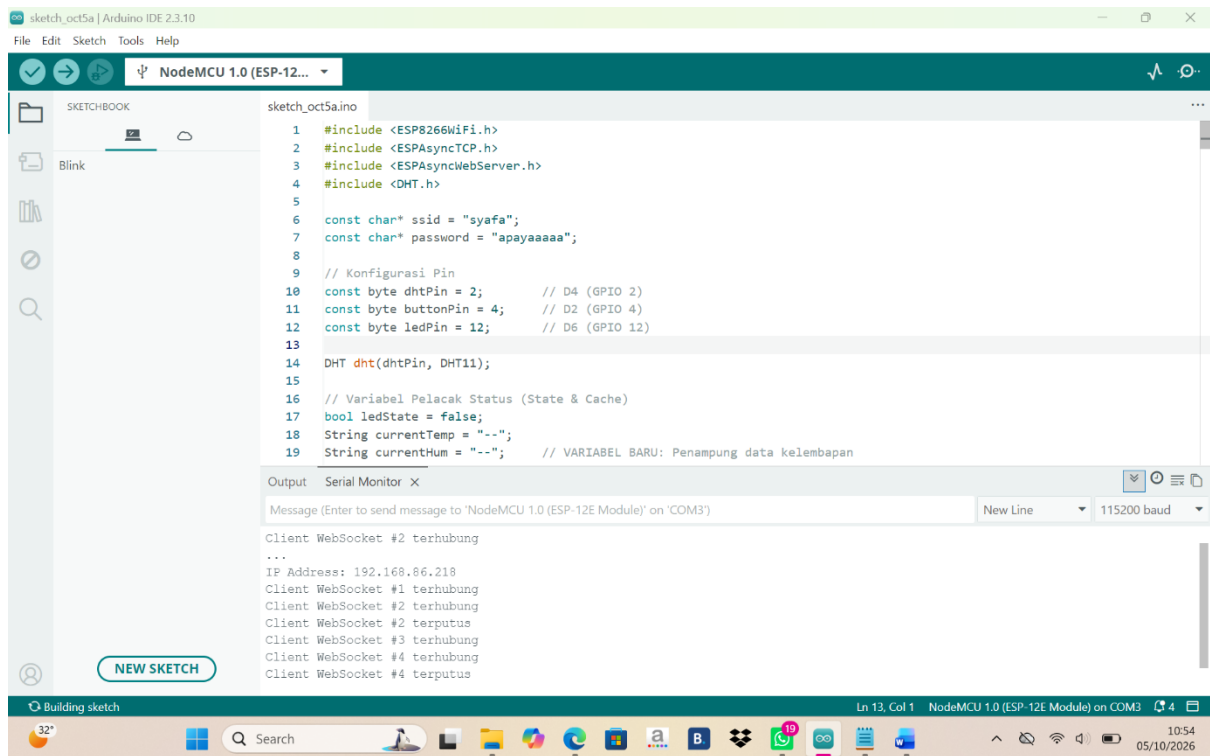
ledStatus — Target ID untuk mengubah teks status perangkat (misalnya menampilkan teks "ON" atau "OFF").

toggleBtn (berupa elemen tombol/button) — Target ID untuk mengubah kelas tampilan CSS (seperti .btn-on atau .btn-off) serta teks pada tombol (seperti "Turn ON" atau "Turn OFF").

Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

- i. **Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--");. Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer *non-blocking* millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).
- ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (<div class="card">) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawaban :



Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

```
Client WebSocket #1 terhubung
Client WebSocket #2 terhubung
...
IP Address: 192.168.86.218
Client WebSocket #1 terhubung
Client WebSocket #2 terhubung
Client WebSocket #2 terputus
Client WebSocket #3 terhubung
Client WebSocket #4 terhubung
```

Kode Modifikasinya :

sketch_oct5a.ino

```
1  #include <ESP8266WiFi.h>
2  #include <ESPAsyncTCP.h>
3  #include <ESPAsyncWebServer.h>
4  #include <DHT.h>
5
6  const char* ssid = "syafa";
7  const char* password = "apayaaaaa";
8
9  // Konfigurasi Pin
10 const byte dhtPin = 2;      // D4 (GPIO 2)
11 const byte buttonPin = 4;   // D2 (GPIO 4)
12 const byte ledPin = 12;     // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT11);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";
19 String currentHum = "--";    // VARIABEL BARU: Penampung data kelembapan
20 int buttonState;
21 int lastButtonState = LOW;
22 unsigned long lastDebounceTime = 0;
23 const unsigned long debounceDelay = 50;
24 unsigned long lastTime = 0;
25
26 // Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
27 AsyncWebServer server(80);
28 AsyncWebSocket ws("/ws");
29
```

sketch_oct5a.ino

```
30 // ----- HTML & JAVASCRIPT (FRONT-END) -----
31 const char index_html[] PROGMEM = R"rawliteral(
32 <!DOCTYPE html>
33 <html>
34 <head>
35   <meta name="viewport" content="width=device-width, initial-scale=1">
36   <title>Real-Time IoT Web</title>
37   <style>
38     body { font-family: Arial; text-align: center; }
39     .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-width: 300px; border-radius: 10px; }
40     button { padding: 15px 30px; font-size: 20px; border-radius: 5px; cursor: pointer; color: white; }
41     .btn-on { background-color: #4CAF50; }
42     .btn-off { background-color: #f44336; }
43   </style>
44 </head>
45 <body>
46   <h1>Smart Room</h1>
47   <div class="card">
48     <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
49   </div>
50
51   <!-- KARTU BARU: Elemen HTML untuk Menampilkan Kelembapan -->
52   <div class="card">
53     <h2>Kelembapan: <span id="humValue">--</span> %</h2>
54   </div>
55
56   <div class="card">
57     <h2>LED: <span id="ledStatus">OFF</span></h2>
58     <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
59   </div>
60
```

sketch_oct5a.ino

```
61 <script>
62 // Membuka terowongan koneksi WebSocket ke alamat IP ESP8266
63 var gateway = `ws://${window.location.hostname}/ws`;
64 var websocket;
65
66 // Menjalankan inisiasi saat halaman pertama kali dimuat
67 window.addEventListener('load', onLoad);
68 function onLoad(event) { initWebSocket(); }
69
70 function initWebSocket() {
71     websocket = new WebSocket(gateway);
72     websocket.onopen = onOpen;
73     websocket.onclose = onClose;
74     websocket.onmessage = onMessage;
75 }
76
77 function onOpen(event) { console.log('WebSocket Terkoneksi'); }
78 function onClose(event) { setTimeout(initWebSocket, 2000); }
79
80 // Dipanggil saat tombol di layar web ditekan
81 function toggleLed(){
82     websocket.send('toggle');
83 }
84
85 // Menangkap paket data JSON yang "didorong" (PUSH) oleh C++ NodeMCU
86 function onMessage(event) {
87     var dataObj = JSON.parse(event.data);
88
89     // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
90     if(dataObj.suhu !== undefined) {
91         document.getElementById('tempValue').innerHTML = dataObj.suhu;
92     }
93 }
```

sketch_oct5a.ino

```
93
94 // MODIFIKASI BARU: Menyuntikkan teks angka Kelembapan ke id "humValue" HTML
95 if(dataObj.hum !== undefined) {
96   document.getElementById('humValue').innerHTML = dataObj.hum;
97 }
98
99 // Merombak UI Tombol & Teks sesuai status hardware
100 if(dataObj.led !== undefined) {
101   var btn = document.getElementById('toggleBtn');
102   var status = document.getElementById('ledStatus');
103   if(dataObj.led == "1"){
104     status.innerHTML = "ON";
105     btn.innerHTML = "Turn OFF";
106     btn.className = "btn-on";
107   } else {
108     status.innerHTML = "OFF";
109     btn.innerHTML = "Turn ON";
110     btn.className = "btn-off";
111   }
112 }
113 }
114 </script>
115 </body>
116 </html>
117 )rawliteral";
118
```

```

119 // ----- BACK-END & WEBSOCKET LOGIC -----
120
121 // MODIFIKASI: Menyisipkan data "hum" ke dalam paket JSON
122 void notifyClients() {
123     String jsonString = "{\"led\": \"\" + String(ledState ? 1 : 0) + "\", ";
124     jsonString += "\"suhu\": \"\" + currentTemp + "\", ";
125     jsonString += "\"hum\": \"\" + currentHum + "\"}"; // Menyertakan kelembapan
126     ws.textAll(jsonString);
127 }
128
129 // Handler pesan masuk (dari klik browser)
130 void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
131     AwsFrameInfo *info = (AwsFrameInfo*)arg;
132     if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT) {
133         data[len] = 0;
134         if (strcmp((char*)data, "toggle") == 0) {
135             ledState = !ledState;
136             notifyClients();
137         }
138     }
139 }
140
141 // Event handler WebSocket bawaan AsyncWebServer
142 void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client, AwsEventType type,
143             void *arg, uint8_t *data, size_t len) {
144     switch (type) {
145         case WS_EVT_CONNECT:
146             Serial.printf("Client WebSocket #%u terhubung\n", client->id());
147             notifyClients(); // Kirim status terkini kepada klien yang baru buka web
148             break;
149         case WS_EVT_DISCONNECT:
150             Serial.printf("Client WebSocket #%u terputus\n", client->id());
151             break;
152         case WS_EVT_DATA:
153             handleWebSocketMessage(arg, data, len);
154             break;
155     }
156 }
157
158 void setup() {
159     Serial.begin(115200);
160
161     // Mengatur pin tombol sebagai input (dengan resistor pull-down eksternal)
162     pinMode(buttonPin, INPUT);
163
164     pinMode(ledPin, OUTPUT);
165     digitalWrite(ledPin, LOW);
166     dht.begin();
167

```

```

167
168   WiFi.mode(WIFI_STA);
169   WiFi.begin(ssid, password);
170   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
171   Serial.println("\nIP Address: " + WiFi.localIP().toString());
172
173   server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
174     request->send_P(200, "text/html", index_html);
175   });
176
177   // Ikat handler WebSockets ke server utama
178   ws.onEvent(onEvent);
179   server.addHandler(&ws);
180   server.begin();
181 }
182
183 void loop() {
184   ws.cleanupClients();
185
186   // 1. Eksekusi perangkat keras LED
187   digitalWrite(ledPin, ledState ? HIGH : LOW);
188
189   // 2. Baca Tombol Fisik (Debounce) tanpa nge-freeze program
190   int reading = digitalRead(buttonPin);
191   if (reading != lastButtonState) {
192     lastDebounceTime = millis();
193   }
194   if ((millis() - lastDebounceTime) > debounceDelay) {
195     if (reading != buttonState) {
196       buttonState = reading;
197
198       // Karena menggunakan resistor PULL-DOWN eksternal, tombol ditekan = HIGH
199       if (buttonState == HIGH) {
200         ledState = !ledState;
201         notifyClients(); // PUSH DATA KE BROWSER SEKETIKA
202       }
203     }
204   }
205   lastButtonState = reading;
206
207   // 3. Baca DHT (Suhu & Kelembapan) setiap 3 detik secara non-blocking
208   if ((millis() - lastTime) > 3000) {
209     float h = dht.readHumidity(); // Baca Kelembapan
210     float t = dht.readTemperature(); // Baca Suhu
211
212     bool dataUpdated = false;
213
214     if(!isnan(h)) {
215       currentHum = String(h);
216       dataUpdated = true;
217     }
218
219     if(!isnan(t)) {
220       currentTemp = String(t);
221       dataUpdated = true;
222     }
223
224     // Jika ada data baru yang valid, push pembaruan ke browser secara otomatis
225     if(dataUpdated) {
226       notifyClients();
227     }
228     lastTime = millis();
229   }
230 }

```

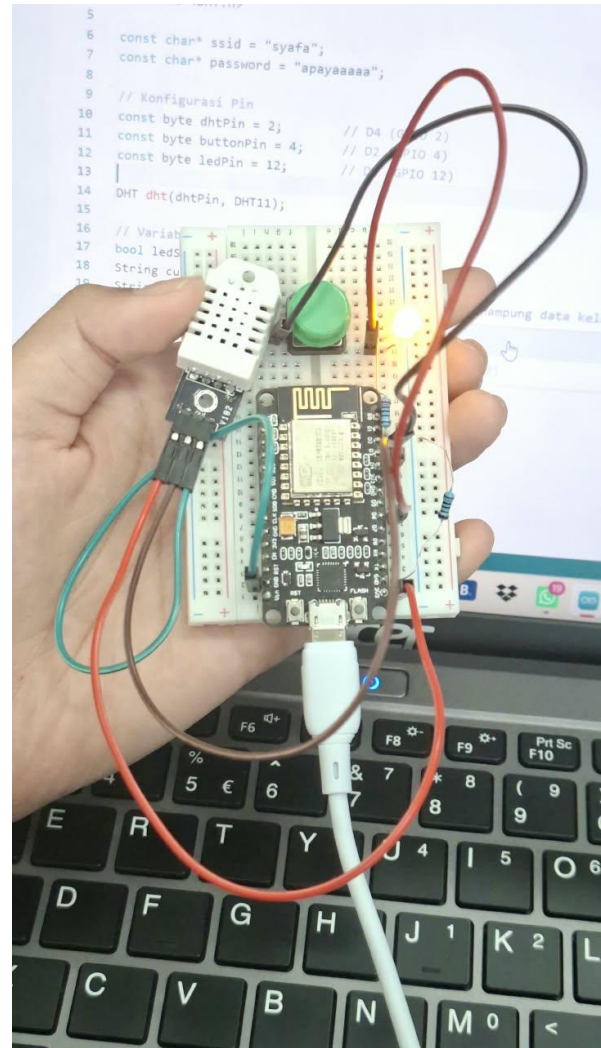
Smart Room

Suhu: 2.40 Celcius

Kelembapan: 23.20 %

LED: ON

Turn OFF



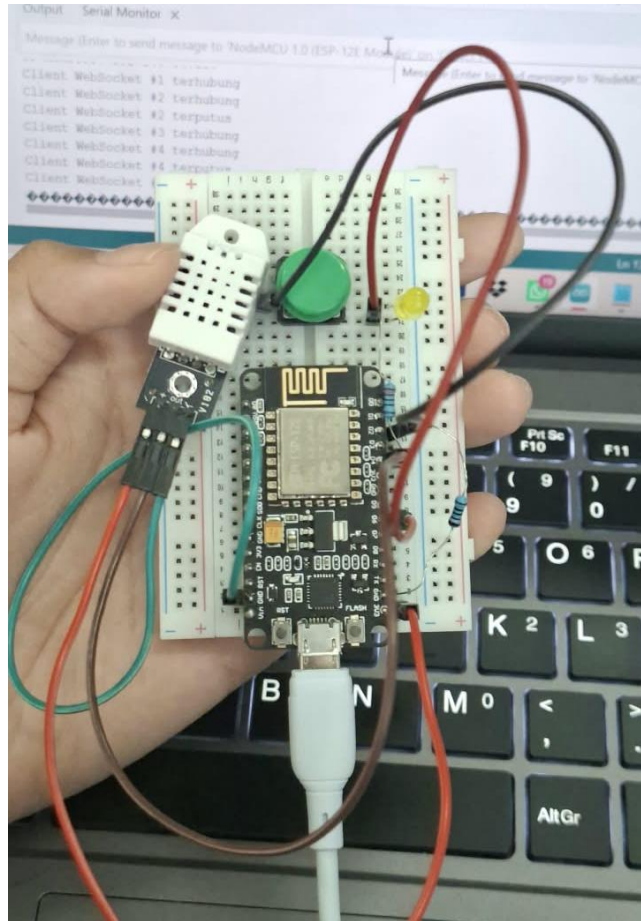
Smart Room

Suhu: 1.80 Celcius

Kelembapan: 17.60 %

LED: OFF

Turn ON



Sisi Back-End (C++ / NodeMCU):

- Menyiapkan variabel penampung string currentHum untuk data kelembapan.
- Membaca nilai kelembapan menggunakan fungsi `dht.readHumidity()` setiap 3 detik secara *non-blocking* dengan `millis()`.
- Mengemas data suhu, kelembapan, dan status LED ke dalam format JSON melalui fungsi `notifyClients()` agar dapat dikirim secara bersamaan.

Sisi Front-End (HTML & JavaScript):

- Menambahkan elemen kartu baru pada halaman web dengan `id="humValue"` sebagai wadah angka kelembapan.
- Menggunakan fungsi JavaScript `onMessage` untuk mengurai data JSON yang masuk dan menyuntikkan nilai kelembapan ke layar secara *real-time* tanpa perlu *refresh* halaman.